

Module 15: Security Hardening

Duration: 3-5 days | **Level:** Advanced

Built-in Laravel Security

Feature	Protection
CSRF tokens	Cross-site request forgery
Password hashing (bcrypt/argon)	Plaintext passwords
Eloquent parameter binding	SQL injection
Blade {{ }} escaping	XSS
Signed URLs	URL tampering
Rate limiting	Brute force

CSRF Protection

All POST/PUT/DELETE forms need `@csrf`. API routes using Sanctum tokens are exempt.

Mass Assignment Protection

```
protected $fillable = ['title', 'description']; // Whitelist
// OR
protected $guarded = ['id', 'is_admin']; // Blacklist
```

Never use `$guarded = []` in production.

Authorization

Always check permissions:

```
$this->authorize('update', $task);
```

Never trust `$request->user_id` from client input.

Environment Security

```
APP_DEBUG=false           # NEVER true in production
APP_ENV=production
```

- Never commit `.env`
 - Rotate `APP_KEY` if compromised
 - Use secrets manager (AWS Secrets Manager, Doppler)
-

HTTPS & Headers

Force HTTPS in AppServiceProvider:

```
if (app()->environment('production')) {
    URL::forceScheme('https');
}
```

```
}
```

Security headers middleware (or nginx):

X-Frame-Options: SAMEORIGIN

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=31536000

Input Validation

Validate ALL user input. Use Form Requests. Sanitize file uploads (type, size, scan).

Dependency Security

composer audit

Enable Dependabot on GitHub for automatic security PRs.

Exercises

1. Add rate limiting to login: throttle:5,1.
 2. Audit TaskForge for mass assignment vulnerabilities.
 3. Configure APP_DEBUG=false and verify stack traces are hidden.
-

Next Module

-> [Module 16: Deployment & Hosting](#)